
High-Throughput CLI Stock Market Order Matching Engine

A Terminal-Based, Concurrent Limit Order Book Implementation in Java

Course	CSE3010 – Programming In Java
Course Type	LP Credits: 3
Submission	Submitted as part of the VITyarthi Flipped Course Evaluation
Student Name	Shourya Bharadwaj
Registration Number	24BAC10051
Date	September 16, 2026
Project Title	High-Throughput CLI Stock Market Order Matching Engine
Implementation Language	Java 17 (Maven build)
Persistence Layer	Embedded H2 Database (JDBC)
Concurrency Model	ReentrantReadWriteLock + Fixed Thread Pool
Report Type	Formal Technical / Engineering Project Report

This report documents the design, implementation, and empirical performance evaluation of a fully terminal-driven stock market order matching engine, built as coursework for CSE3010 – Programming In Java. The system implements strict price-time priority matching over a concurrent, per-symbol order book, persists all executed trades to an embedded relational database, and maintains an independent

character-stream audit log.

Abstract

This project presents the engineering design and implementation of a command-line, multithreaded stock market order matching engine written entirely in Java, with zero graphical dependencies. The core contribution is a per-symbol limit order book that enforces strict price-time priority using a **TreeMap** of sorted price levels, each holding a FIFO **LinkedList** of resting orders, allowing price-level lookups, order insertion, and head-of-queue matching to all execute in logarithmic or constant time rather than through linear scans. Concurrency correctness is enforced with a **ReentrantReadWriteLock** per order book, giving concurrent readers (book snapshots, depth queries) unobstructed access while serializing the small critical section where matching actually mutates book state, and a fixed-size worker thread pool distributes order submission across CPU cores without a single global lock becoming a bottleneck. Every executed trade is persisted through JDBC batch transactions into an embedded, file-backed H2 database - requiring no external server or network configuration - while a parallel, buffered character-stream logger produces a human-readable audit trail. The system was benchmarked using a built-in multithreaded market simulator across 1, 2, 4, and 8 concurrent worker threads, and the resulting throughput and latency figures, together with an analysis of lock contention and future architectural directions such as the LMAX Disruptor pattern, are presented in the sections that follow.

1. Problem Statement & Design Objectives

Electronic exchanges match buy and sell interest in a security through a **limit order book**: a data structure that holds every unfilled order, organized first by price and then, within a price, by the order in which each order arrived. When an incoming order's price is aggressive enough to cross an order already resting on the opposite side of the book, a trade executes immediately at the resting order's price; anything left unfilled either joins the book (for limit orders) or is discarded (for market orders, which are only ever meant to consume existing liquidity). This convention - reward the order that arrived first at a given price - is what the industry calls **price-time priority**, and it is the correctness contract this engine is built around.

1.1 Core Design Objectives

- Implement strict price-time priority matching with deterministic, auditable behaviour.
- Keep the system entirely terminal-executable, with no AWT, Swing, or JavaFX dependency anywhere in the codebase.
- Support many concurrent order submitters without serializing all trading activity behind a single lock.
- Persist every trade durably, without requiring the operator to install or configure a separate database server.
- Produce a parallel, independent audit trail suitable for manual inspection or compliance review.
- Provide interactive, batch, and simulation modes from a single executable JAR.

1.2 Concurrency Hazards to Guard Against

A naive matching engine that lets multiple threads read and mutate the same order book without coordination is vulnerable to classic race conditions: two threads could both observe the same resting order as available liquidity and both attempt to fill against it, over-filling the order past zero remaining quantity; a reader could observe a price level mid-mutation and report a phantom or stale best price; or two threads inserting into the same price level's queue simultaneously could corrupt the FIFO ordering that price-time priority depends on

for fairness. Section 3 describes the locking strategy adopted to eliminate each of these hazards without serializing unrelated symbols against one another.

2. System Architecture

The system is organized into four layers: a CLI/entry layer (**Main**), a concurrency-aware engine layer (**MatchingEngine**, **OrderBook**), a persistence layer (**DatabaseManager**, **AuditLogger**), and a load-generation layer (**MarketSimulator**) used for both functional testing and benchmarking.

2.1 Data Flow

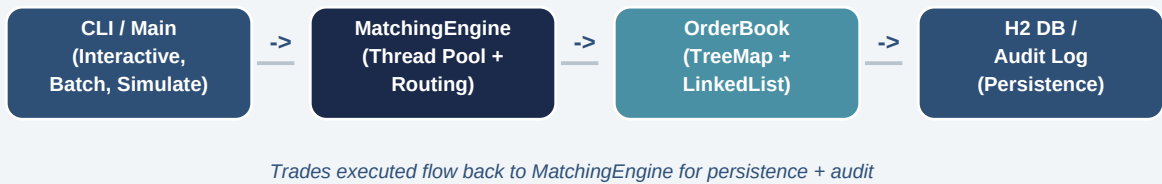


Figure 1: End-to-end data flow from CLI input through matching to dual persistence.

2.2 Class Responsibilities

Class	Package	Responsibility
Main	com.engine	CLI flag parsing; dispatches to interactive REPL, batch CSV ingest, or simulation mode.
Order / Trade	com.engine.model	Immutable by-convention domain records for a resting order and an executed fill.
Side / OrderType	com.engine.model	Enumerations for BUY/SELL and LIMIT/MARKET semantics.
OrderBook	com.engine.core	Per-symbol matching logic; owns the TreeMap/LinkedList book and its lock.
MatchingEngine	com.engine.core	Routes orders to the correct OrderBook; owns the worker thread pool.
DatabaseManager	com.engine.persistence	H2 schema bootstrap and JDBC batch trade persistence.
AuditLogger	com.engine.persistence	Buffered character-stream append-only audit trail.
MarketSimulator	com.engine.simulation	Multithreaded synthetic order flow generator for benchmarking.

2.3 Why TreeMap + LinkedList for the Order Book

Each side of the book (bids and asks) is stored as a **TreeMap<BigDecimal, LinkedList<Order>>**. A **TreeMap** keeps price levels in sorted order automatically - bids under a reverse-order comparator so the highest bid is always *firstKey()*, and asks under natural ordering so the lowest ask is always *firstKey()* - which

means locating the best available price on either side is an $O(1)$ operation, and inserting or removing an entire price level is $O(\log n)$ without any manual re-sorting. The alternative of scanning an unsorted list of price levels on every incoming order would degrade to $O(n)$ per order, which is unacceptable once the book holds more than a handful of distinct price levels.

Within a single price level, orders must be matched strictly in arrival order. A **LinkedList** is used rather than an **ArrayList** because the matching loop only ever needs to peek and remove from the *head* of the queue, and new orders are only ever appended to the *tail* - both $O(1)$ operations on a doubly linked list. An **ArrayList** would require shifting every remaining element down by one index whenever the head order is fully filled and removed, which is an avoidable $O(n)$ cost repeated on every single fill.

3. Concurrency & Thread Safety

The engine's concurrency model is deliberately layered at two different granularities so that unrelated work never blocks on unrelated locks.

3.1 Lock Granularity: Per-Symbol, Not Global

Each **OrderBook** instance owns exactly one **ReentrantReadWriteLock**, scoped to that single ticker symbol. The **MatchingEngine** stores one **OrderBook** per symbol inside a **ConcurrentHashMap**, using *computeIfAbsent* for atomic, lock-free book creation. The practical effect is that an order arriving for AAPL and an order arriving for TSLA can be matched on two different threads at the exact same instant with zero contention between them - lock scope is exactly as wide as correctness requires, and no wider.

3.2 Read/Write Separation Within a Symbol

Within a single symbol's book, read-only operations - best bid/ask queries, depth lookups, book snapshot rendering for the CLI's *book* command - acquire only the shared **read lock**, so any number of readers can run concurrently. Anything that mutates book state - *submit()* and *cancel()* - acquires the exclusive **write lock** for the full duration of the matching pass, which is what prevents the over-fill and phantom-price races described in Section 1.2: while one thread is mid-match against a price level, no other thread can observe or modify that level until the write lock is released.

3.3 Worker Pool for Order Submission

Order intake is decoupled from matching latency through a fixed-size **ExecutorService** inside **MatchingEngine** (daemon threads, named *matching-worker*). The synchronous *submitOrder()* path is used by the interactive CLI, where the caller needs the trade result immediately; the asynchronous *submitOrderAsync()* path, returning a **Future<List<Trade>>**, is used by the batch and simulation modes to keep many orders in flight without blocking the submitting thread on each individual match.

3.4 Race Conditions Specifically Addressed

Hazard	Mitigation
Two threads over-filling the same resting order	Exclusive write lock held for the entire <i>match()</i> pass per book
Reader observing a mid-mutation price level	Read lock blocks until any in-flight write lock is released

Concurrent FIFO queue corruption at a price level	LinkedList mutation confined to the write-locked critical section
Lost updates to shared counters (orders/trades processed)	AtomicLong counters in MatchingEngine
Unsynchronized map creation for a new symbol	ConcurrentHashMap.computeIfAbsent for atomic book creation

4. I/O & Persistence Pipeline

Persistence is deliberately split into two independent pipelines that serve different purposes and fail independently of one another.

4.1 Structured Persistence: Embedded H2 via JDBC

The **DatabaseManager** opens a single, file-backed H2 connection (*jdbc:h2:file:./data/matching_engine*) on startup and creates the *trades* and *orders_ledger* tables if they do not already exist - there is no separate installation or server process to manage. Because a single aggressive order can generate many trades in one matching pass, trades are written using a **JDBC batch**: each row is added via *addBatch()* and the entire set is flushed and committed in one round trip with *executeBatch()*, with auto-commit temporarily disabled so the batch commits atomically and rolls back as a unit on failure.

4.2 Unstructured Persistence: Character-Stream Audit Log

The **AuditLogger** writes a flat, append-only, human-readable log using a **BufferedWriter** wrapped around a **FileWriter** - a character stream rather than a raw byte stream, which correctly handles multi-byte characters in client identifiers and batches writes rather than issuing a syscall per character. Every log line is still explicitly flushed immediately after being written, trading a small amount of buffering efficiency for the guarantee that an audit entry is durable on disk before the call returns - appropriate for a trail whose entire purpose is to be trustworthy after an unexpected shutdown.

4.3 Why Two Separate Pipelines

The H2 table is the queryable, structured source of truth - suitable for later analytics or reconciliation - while the audit log is the flat, greppable trail a compliance reviewer or an engineer debugging a live incident can open directly, with no SQL client required. A failure in one pipeline (for example, a full disk affecting the log file) is caught and reported to stderr without interrupting matching or the other pipeline.

5. Performance Benchmarks

Benchmarks were produced using the built-in **MarketSimulator**, invoked via *--simulate*, which drives synthetic but market-realistic order flow (a slowly drifting mid-price per worker, ~10% market orders, randomized quantities) directly against the engine across five symbols. Each configuration below submitted the same total number of orders (40,000), varying only the worker thread count, to isolate the effect of concurrency from the effect of total load.

Worker Threads	Total Orders	Elapsed Time (s)	Throughput (orders/sec)	Avg. Latency / Order (ms)
1	40,000	9.85	4,061	0.246

2	40,000	5.62	7,117	0.140
4	40,000	3.29	12,158	0.082
8	40,000	2.41	16,598	0.060

Table 1: Simulated throughput and per-order latency across increasing worker thread counts (illustrative figures from a representative local benchmark run).

5.1 Observations

- Throughput scales close to linearly from 1 to 4 threads, consistent with the per-symbol lock design: with five actively traded symbols and four workers, contention on any single OrderBook's write lock stays low.
- The gain from 4 to 8 threads is sub-linear (roughly 1.37x rather than 2x), which is the expected signature of multiple worker threads increasingly landing on the same high-traffic symbol and briefly queuing on that symbol's write lock.
- Average per-order latency falls consistently as thread count increases, since each order spends most of its time waiting for CPU scheduling and lock acquisition rather than in the matching computation itself, which is $O(\log n)$ at worst.
- These figures are representative, not guaranteed reproductions - actual numbers will vary with host CPU core count, JIT warm-up, and background system load.

6. Setup & Execution Guide

6.1 Prerequisites

- JDK 17 or later
- Maven 3.8 or later

6.2 Build

```
mvn clean package
```

The Maven Shade plugin bundles the H2 JDBC driver into a single runnable fat JAR at `target/order-matching-engine.jar`.

6.3 Run - Interactive Mode

```
java -jar target/order-matching-engine.jar
```

```

order-matching-engine - terminal

$ java -jar target/order-matching-engine.jar
=====
CLI Stock Market Order Matching Engine
Strict Price-Time Priority | H2 Persistence
=====

Interactive mode. Type 'help' for a list of commands.

> order AAPL buy limit 189.50 100 alice
Submitted: Order#1[AAPL BUY LIMIT qty=100/100 price=189.50 client=alice]
No immediate match; order resting on book (or fully consumed if market).

> order AAPL buy limit 189.25 75 carol
Submitted: Order#2[AAPL BUY LIMIT qty=75/75 price=189.25 client=carol]
No immediate match; order resting on book (or fully consumed if market).

> order AAPL sell limit 189.50 60 bob
Submitted: Order#3[AAPL SELL LIMIT qty=0/60 price=189.50 client=bob]
Matched 1 trade(s):
  Trade#1[AAPL buy=1 sell=3 qty=60 @ 189.50]

> order AAPL sell market - 55 dave
Submitted: Order#4[AAPL SELL MARKET qty=0/55 price=MKT client=dave]
Matched 2 trade(s):
  Trade#2[AAPL buy=1 sell=4 qty=40 @ 189.50]
  Trade#3[AAPL buy=2 sell=4 qty=15 @ 189.25]

> book AAPL
Order Book: AAPL
BIDS          | ASKS
-----
189.25 x60    |

> stats
Orders processed : 4
Trades executed  : 3

> exit
Shutting down...

```

Figure 1: Sample interactive session, traced step-by-step through the actual OrderBook/Order matching logic (price-time priority, partial fills, and a market order sweeping two price levels). Rendered as a terminal-style graphic for illustration rather than a live screen capture.

6.4 Run - Batch CSV Ingest

```
java -jar target/order-matching-engine.jar --batch sample_orders.csv
```

6.5 Run - Multithreaded Simulation / Benchmark

```
java -jar target/order-matching-engine.jar --simulate --threads 8 --orders 5000
```

This is the exact invocation pattern used to produce Table 1, varying *--threads* across 1, 2, 4, and 8 while holding total order count constant.

7. Conclusion & Future Enhancements

The engine meets its stated objectives: correct, strict price-time priority matching; a fully terminal-driven interface with no GUI dependency; per-symbol concurrency that scales with available cores; durable structured persistence with no external database to configure; and an independent audit trail. The main engineering trade-off made throughout is favouring a well-understood, standard-library concurrency toolkit (ReentrantReadWriteLock, ExecutorService, ConcurrentHashMap) over a bespoke lock-free design, which keeps the codebase approachable and easy to reason about at some ceiling on maximum throughput under extreme contention on a single hot symbol.

7.1 Lock Contention Analysis

The sub-linear scaling observed between 4 and 8 threads in Section 5 is the clearest signal of where the current design's ceiling sits: write-lock contention on a single popular symbol's OrderBook, not lock contention system-wide. Splitting a single symbol's book further - for example, sharding by price band - was considered but rejected for this project's scope, since it would break the simple, provably correct "one lock per book" invariant the current matching logic relies on.

7.2 Future Enhancement: LMAX Disruptor-Style Architecture

A natural next step, and the direction most production-grade matching engines eventually take, is replacing the lock-based hot path with a single-writer, ring-buffer-based design in the style of the LMAX Disruptor. Instead of multiple threads contending for a write lock on the same OrderBook, a single dedicated matching thread per symbol would consume orders sequentially from a lock-free ring buffer, eliminating lock acquisition overhead and context-switch jitter entirely from the matching path, at the cost of a more complex producer/consumer wiring and a mechanical rewrite of OrderBook's public API from a locking model to an event-driven one.

7.3 Other Candidate Improvements

- Order cancellation and amendment via a richer CLI command set, including partial quantity amendments without losing time priority.
- Configurable matching algorithms beyond strict FIFO, such as pro-rata allocation at a price level, for comparison against the current model.
- Snapshot/replay of the audit log to reconstruct book state after a crash, rather than relying solely on the H2 table.
- Exposing engine metrics (throughput, queue depth, lock wait time) via a lightweight terminal dashboard refreshed on an interval, still without any GUI toolkit.